

Unidad 6 — El pensamiento computacional como estrategia para la solución de problemas

TECNOLOGÍA • AÑO 1 • SEMESTRE 2 • ÁMBITO: PENSAMIENTO COMPUTACIONAL

Objetivo de la unidad

Diseñar algoritmos y programas de complejidad intermedia para resolver problemas contextualizados, empleando lenguajes de programación estructurada e IA generativa como apoyo crítico.

1. Los 4 pilares del pensamiento computacional

Pilar	Qué significa	Ejemplo
Descomposición	Dividir un problema grande en partes más pequeñas y manejables	Para "organizar una fiesta de la escuela": lista de invitados, presupuesto, decoración, comida
Reconocimiento de patrones	Encontrar semejanzas o repeticiones entre problemas	Notar que "calcular el promedio de exámenes" y "calcular el promedio de asistencias" usan la misma lógica
Abstracción	Quedarse con lo esencial, ignorando detalles irrelevantes	Un mapa del metro no muestra la forma real de las calles, solo las conexiones entre estaciones
Algoritmos	Una secuencia ordenada y precisa de pasos para resolver un problema	Una receta de cocina, o un procedimiento matemático

2. Algoritmos y diagramas de flujo

Un **algoritmo** se puede representar en pseudocódigo (texto estructurado) o en un **diagrama de flujo**, usando símbolos estándar:

Símbolo	Significado
Óvalo	Inicio / Fin
Rectángulo	Proceso o instrucción
Rombo	Decisión (pregunta con respuesta Sí/No)
Paralelogramo	Entrada / Salida de datos
Flechas	Orden de ejecución

3. Estructuras de control

- **Secuencia:** las instrucciones se ejecutan una tras otra, en orden.
- **Condicional (Si / Si no):** ejecuta un bloque u otro según si se cumple una condición.
- **Ciclo o iteración (Mientras / Para):** repite un bloque de instrucciones mientras se cumpla una condición, o un número determinado de veces.

4. Ejemplo resuelto — algoritmo con condicional

Problema: dado tres números, encontrar el mayor.

```
Inicio
  Leer a, b, c
  Si a > b entonces
    mayor ← a
  Si no
    mayor ← b
  Fin Si
  Si c > mayor entonces
    mayor ← c
  Fin Si
  Escribir mayor
Fin
```

Verificación con a=3, b=7, c=5: ¿3>7? No → mayor=7. ¿5>7? No → mayor sigue siendo 7. **Resultado: 7** ✓ (correcto, 7 es el mayor de los tres).

5. Ejemplo resuelto — algoritmo con ciclo

Problema: contar cuántos números pares hay entre 1 y N.

```
Inicio
  Leer N
  contador ← 0
  Para i ← 1 hasta N hacer
    Si i mod 2 = 0 entonces
      contador ← contador + 1
    Fin Si
  Fin Para
  Escribir contador
Fin
```

Verificación con N=10: i recorre 1,2,...,10; son pares 2,4,6,8,10 → el ciclo suma 1 al contador cada vez que i es par. **Resultado: contador = 5** ✓ (coincide con los 5 números pares entre 1 y 10).

6. Tipos de fallas (errores) en un programa

- **Falla de sintaxis:** se rompe una regla del lenguaje de programación (ej. falta un paréntesis). La computadora **sí** la detecta y detiene la ejecución.
- **Falla de lógica:** el programa se ejecuta sin errores, pero el resultado es incorrecto porque el algoritmo estaba mal planteado (ej. usar `>` cuando debía ser `>=`). La computadora **no** la detecta por sí sola — hay que revisar los resultados.

7. Uso de la IA generativa para programar

- Puede ayudar a generar una primera versión del código o a explicar un mensaje de error.
 - **Siempre debe probarse con casos de prueba propios** antes de confiar en que el código funciona correctamente (la IA también puede introducir fallas de lógica).
-

Errores comunes

- Confundir falla de sintaxis (la detecta el compilador/intérprete) con falla de lógica (hay que revisarla uno mismo, comparando el resultado esperado contra el obtenido).
- Escribir un algoritmo sin antes probarlo "a mano" con un ejemplo concreto (como se hizo arriba).
- Aceptar código generado por IA sin probarlo con al menos 2-3 casos distintos.

Resumen rápido

- 4 pilares: descomposición, reconocimiento de patrones, abstracción, algoritmos.
- Estructuras de control: secuencia, condicional, ciclo.
- Antes de dar un algoritmo por bueno, Pruébalo a mano con un ejemplo y verifica el resultado.
- Falla de sintaxis → la detecta la computadora; falla de lógica → hay que revisarla tú.